Computer Science & IT



Data Structure & Programming

Linked List

Lecture No. 01



By- Abhishek Sir

Recap of Previous Lecture



- Topic
- Topic
- Topic
- Topic
- Topic

Circular queue

Stack implementation of queue

Slide

Topics to be Covered



Topic	Linked list
Topic	Structure
Topic	"." & → operator
Topic	Dynamic Memory allocation
Topic	Creating a Node, creating a list

Slide



Topic : Deque & Priority Queue

Deque : Doubly ended queue Insertion & deletion can be performed both end.

- Insert rear
- Delek front
- Delek rear
- Insert fount

Slide



Topic : Deque & Priority Queue

priority queue :- Element can be inserted in priority queue in any but delete will happen based on priority

one way of implementation of priority queue is using binary Heap

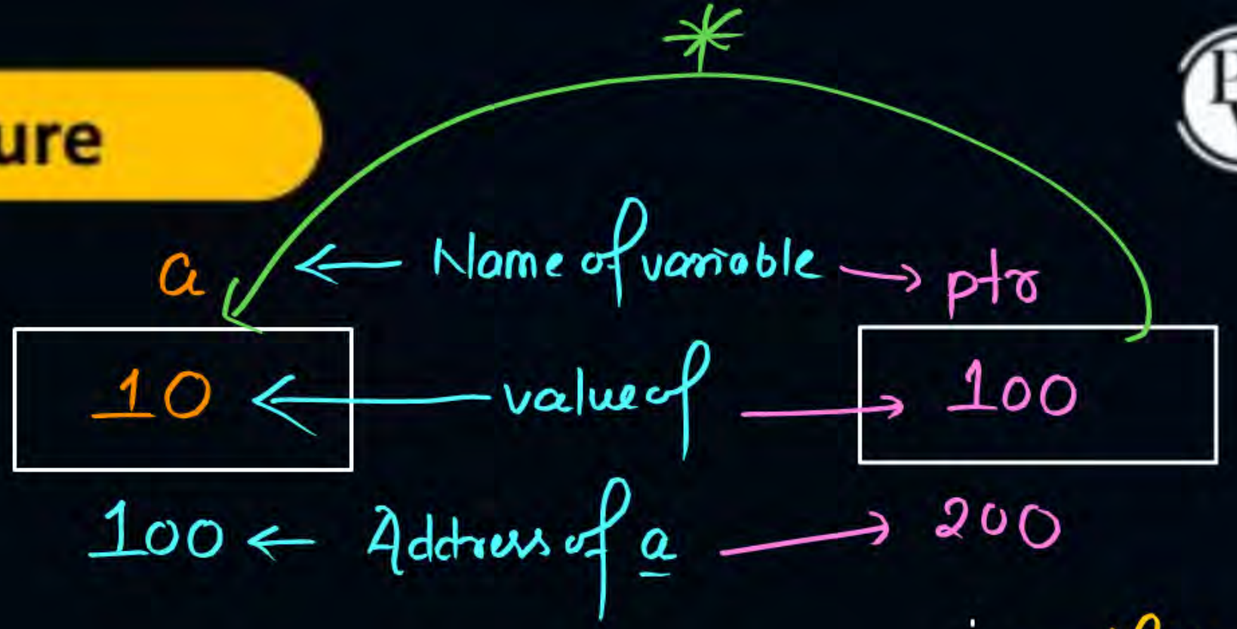
- Max heap
- Min Heap

} Application -: Greedy Algorithm

Slide

Topic : Pointer & Structure





int a; ←

a = 10

variable - pointer variable

pointer variable can hold address of another variable of same data type.

int *ptr;

ptr = &a;

a = a+10

printf("%u", ptr);

100

printf("%d", *ptr);

10

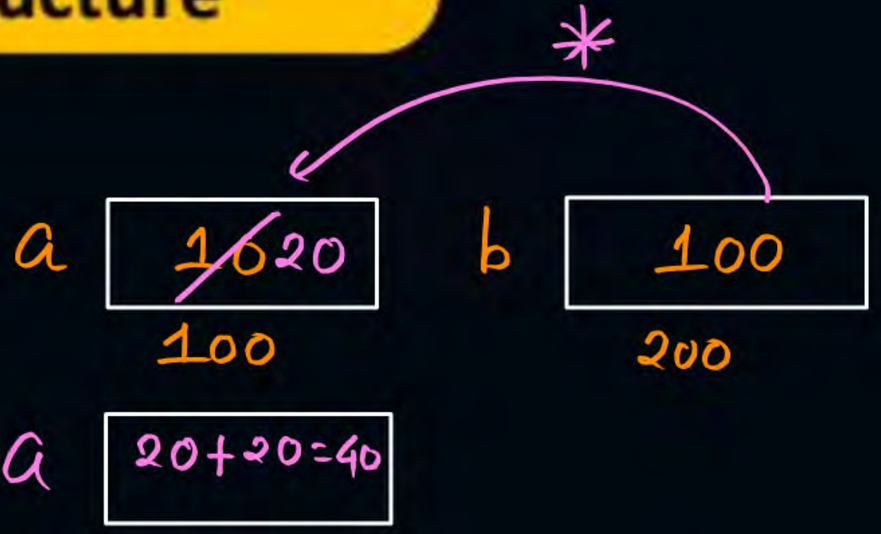
Slide

Topic : Pointer & Structure

```
#include<stdio.h>
int main (){
        int a, *b;
        a = 10;
        b = &a;
        a =a+10;
        *b = *b+20;
        printf("%d", a);
        return 0;
}
```

read

write



40

Slide



Topic : Pointer & Structure

p=100, 20 | p=200, 30 | 300, 20

```
#include<stdio.h>
void foo(int*p, int val){
    *p = val;
}
int main (){
    int a, b,c;
    foo(&a, 20);
    foo(&b, a+10);
    foo(&c, b-10);
    printf("%d", a+b+c);
    return 0;
}
```



20+30+20 = 70

Slide

GATE 2017

$x << k = x * 2^k$

$x >> k = \left\lfloor \frac{x}{2^k} \right\rfloor$

#Q Consider the following program

```
#include<stdio.h>
void foo(int*p, int val){
    *p = val;
}
void bar (int x, int *y) {
    *y =x<<2;
}
int main (){
    int a,b,c;
    foo(&a, 20);
    bar(a, &b);
    bar(b, &c);
    printf("%d", a+b+c);
    return 0;
}
```

Output of the program is

420

p=100, 20

a 20 (100) b 80 (200) c 320 (300)

(1) bar 20 y 200

= 20<<2 = 20×2² = 20×4 = 80

(2) x=80 y 300

80×2² = 320

Slide



Topic : Pointer & Structure

Structure : user defined data type

Structure is collection of dissimilar Data type.

```
struct test {
    int a;
    char ch;
};
```

Members of Structure

Structure variable

```
struct test t = { 14, 'a' };
```

To access structure member "." operator is used.

```
printf ("%d", t.a);
printf ("%c", t.ch);
```

Slide



Topic : Pointer & Structure

→ dereference

Structure : user defined data type

Structure is collection of dissimilar Data type.

```
struct test {
    int a;
    char ch;
};
```

Members of Structure

compiler

```
struct test t = { 14, 'a' };

struct test *ptr;

ptr = &t;
```

To access structure member using structure pointer

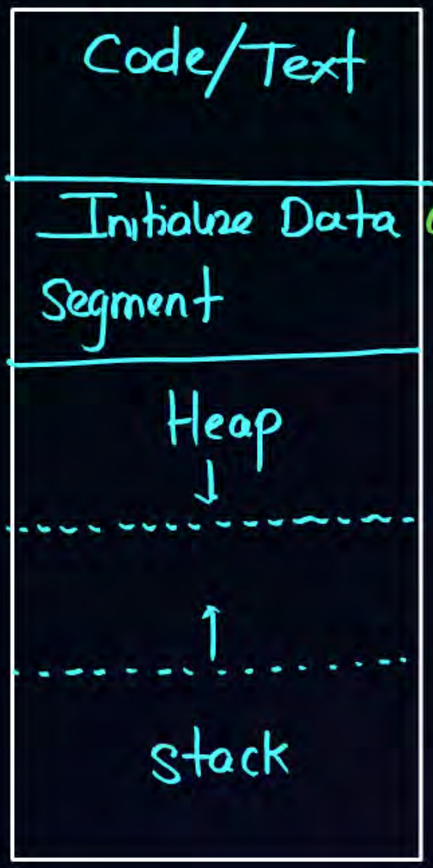
→ operator

```
printf ("%d", ptr->a);
printf ("%c", ptr->ch);
```

Slide



Topic : Dynamic Memory Allocation



global / Local

Compile time allocation

During run time if memory required to be allocated.

Dynamic allocation of Memory is done

* malloc ( )
* calloc ( )

Deallocation

free ( )

Slide



Topic : Dynamic Memory Allocation

malloc & calloc allocates memory in Heap Area & return a pointer of first location

```
int *ptr=(int*)malloc(sizeof(int));
```

malloc return a void pointer

typecast

void pointer convert to integer pointer



100

```
int *ptr = malloc(sizeof(int));
```

Internaly type casting done

No Error or warning

Slide



Topic : Dynamic Memory Allocation

ptr 100

*ptr = 40;



100

deallocate the memory

free(ptr);

if memory allocated and not deallocated then it leads to memory Leak

Slide



Topic : Dynamic Memory Allocation

* Calloc() : contiguous allocation

Calloc (Sizeof(datatype), No. of allocation)

Calloc (Sizeof(int), 10);

2 coguments

All memory Location

Initialize with 0

Slide

Q. Suppose a stack implementation supports an instruction REVERSE, which reverses the order of elements on the stack, in addition to the PUSH and POP instructions. Which one of the following statements is TRUE with respect to this modified stack?

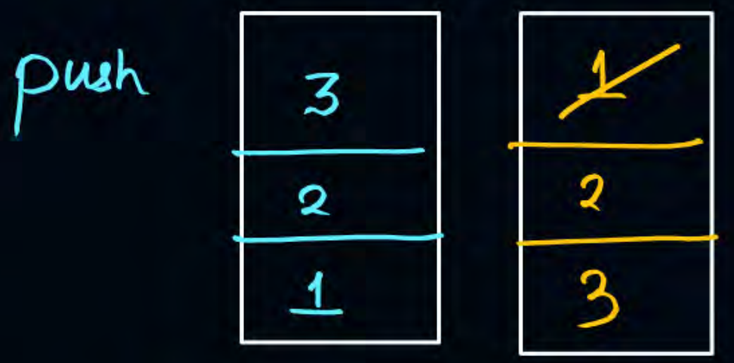
(A) A queue cannot be implemented using this stack. ✗

(B) A queue can be implemented where ENQUEUE takes a single instruction and DEQUEUE takes a sequence of two instructions. → Restore

(C) A queue can be implemented where ENQUEUE takes a sequence of three instructions and DEQUEUE takes a single instruction.

(D) A queue can be implemented where both ENQUEUE and DEQUEUE take a single instruction ✗

(GATE_2014_2 M)



Enqueue - push

dequeue - 2 Ins.

reverse po

Enqueue(1), Enqueue(2) Enqueue

dequeue reverse pop

Enqueue(4)

Q. Suppose a stack implementation supports an instruction REVERSE, which reverses the order of elements on the stack, in addition to the PUSH and POP instructions. Which one of the following statements is TRUE with respect to this modified stack?

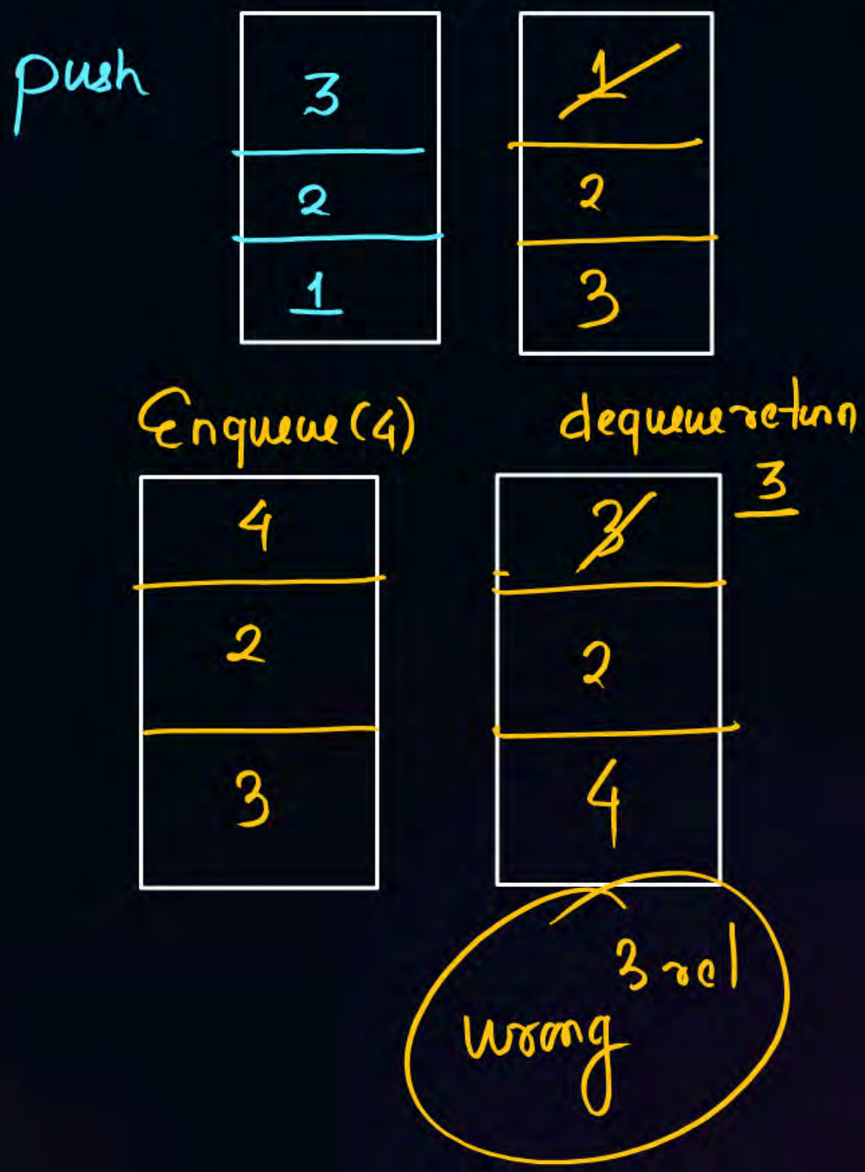
(A) A queue cannot be implemented using this stack. ✗

(B) A queue can be implemented where ENQUEUE takes a single instruction and DEQUEUE takes a sequence of two instructions.

(C) A queue can be implemented where ENQUEUE takes a sequence of three instructions and DEQUEUE takes a single instruction.

(D) A queue can be implemented where both ENQUEUE and DEQUEUE take a single instruction

(GATE_2014_2 M)



Q. Suppose a stack implementation supports an instruction REVERSE, which reverses the order of elements on the stack, in addition to the PUSH and POP instructions. Which one of the following statements is TRUE with respect to this modified stack?

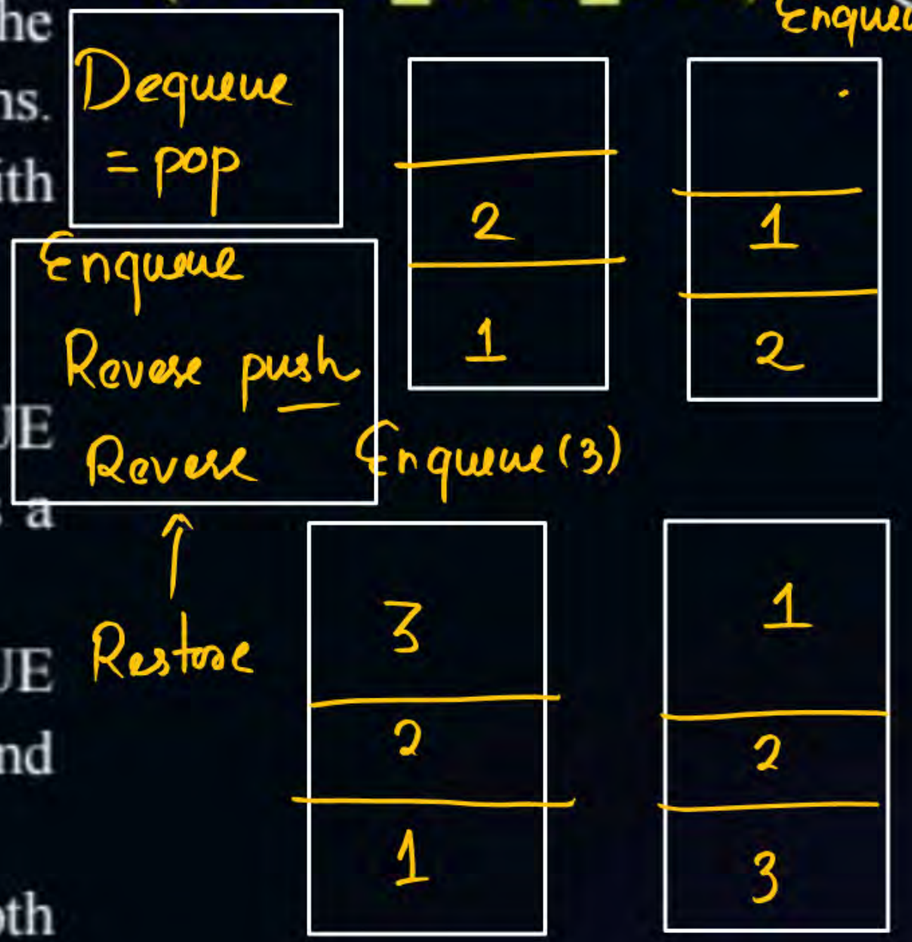
(A) A queue cannot be implemented using this stack.

(B) A queue can be implemented where ENQUEUE takes a single instruction and DEQUEUE takes a sequence of two instructions.

(C) A queue can be implemented where ENQUEUE takes a sequence of three instructions and DEQUEUE takes a single instruction.

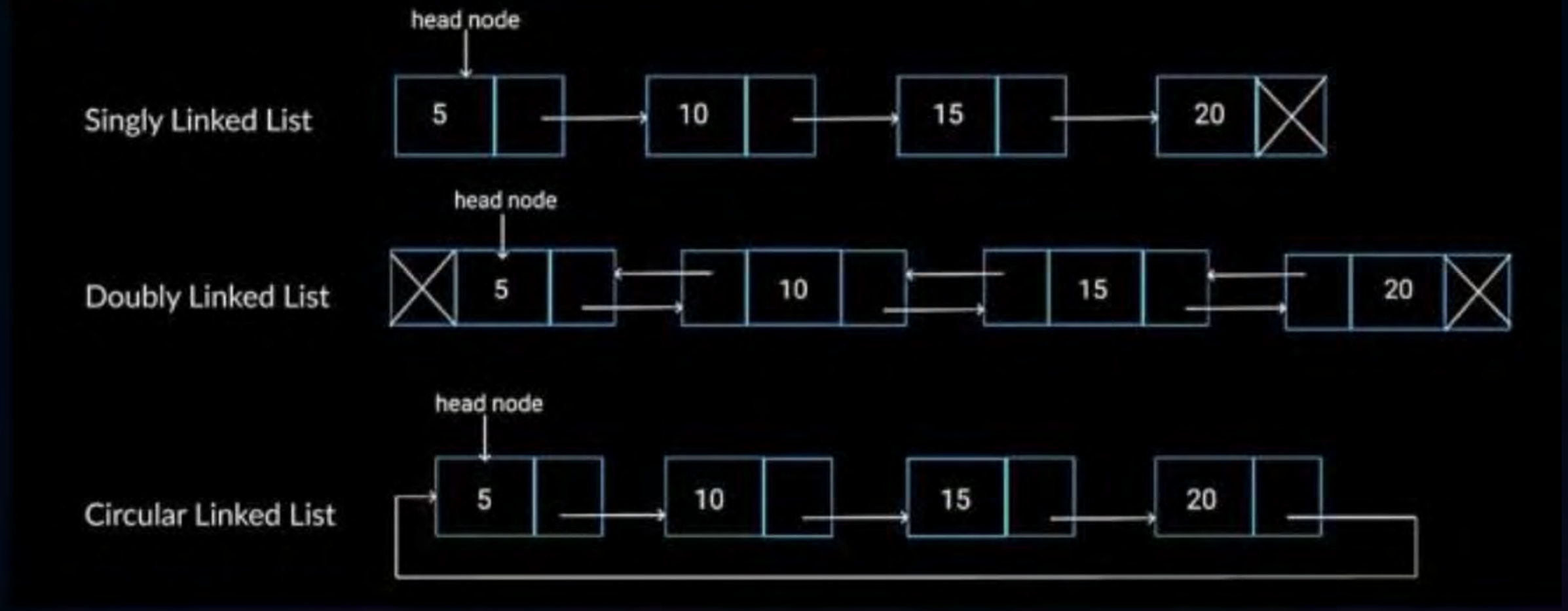
(D) A queue can be implemented where both ENQUEUE and DEQUEUE take a single instruction

(GATE_2014_2 M)





Topic : Linked List





Topic : Linked List

Singly Linked List

Code ↑

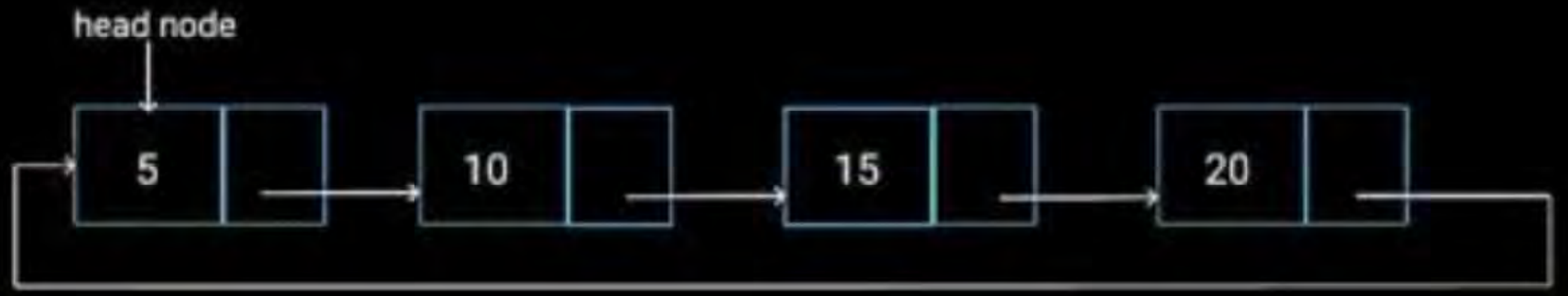


Doubly Linked List



Complexity

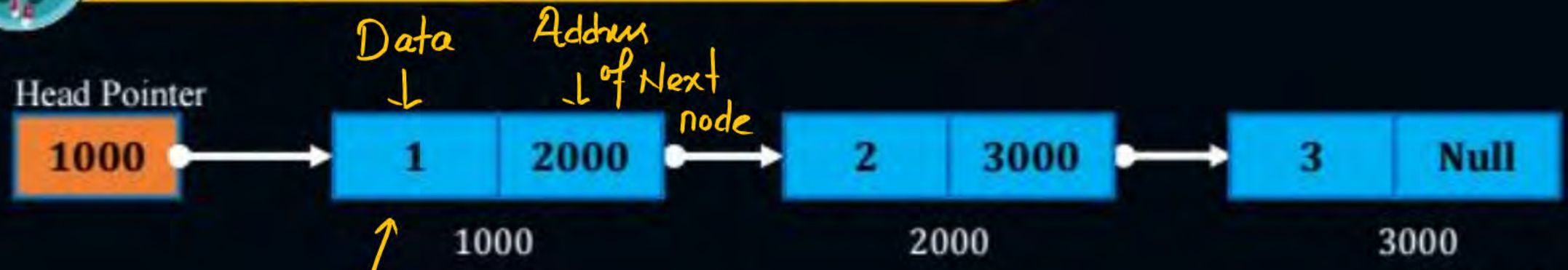
Circular Linked List



Slide



Topic : Single Linked List



Node of Linked List

```
typedef struct node {
    int data;
    struct node * next;    // * link
} node;
```

```
struct node p;
node p;
```

Slide

Topic : Single Linked List



node* getnode (int)

getnode function creates a linked list node (Single) and assign x to data part & NULL to next part & return address of Node (Dynamic)

```
# include<stdlib.h>
node * getnode (int x){
node *ptr;
ptr = malloc ( sizeof(node));
ptr→data = x;
ptr→ next = NULL;
return ptr;
}
```

ptr 1000

Slide



Topic : Single Linked List



$temp_1 = 1000$ | $temp_2 = 2000$ | $temp_3 = 3000$

node* Build123()

Build123 create

a Single linked list

1 → 2 → 3

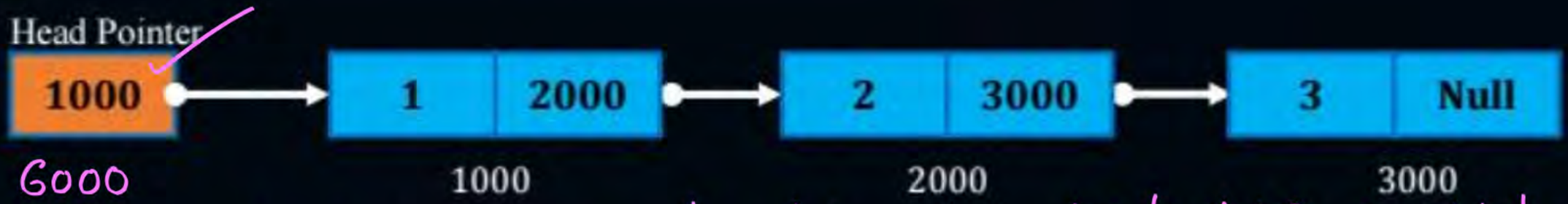
and return address of

first Node

```
node* Build123(){
    node: *temp1, *temp2, *temp3;
    temp1 = getnode(1);
    temp2 = getnode(2);
    temp3 = getnode(3);
    temp1->next = temp2;
    temp2->next = temp3;
}return temp1;
```

Slide

Topic : Single Linked List



The entire Linked is maintained by a pointer to first Node of list called Head pointer.

```
node *HP; ← global variable

int main(){

    HP = build123();
    print(HP);
    return 0;
}
```

Slide

Topic : Single Linked List



S != NULL

```
void print( node* s){
    while(S != NULL) {
        printf("%d", s->data);
        S = S->next;
    }
}
```

S 1000
1000 != NULL
1
S = 2000

S 2000
2000 != NULL
2
S = 3000

3000 != NULL
3
S NULL
NULL != NULL

Slide



2 mins Summary

Topic — pointer

Topic — structure ". ' " → "

Topic — Dynamic Memory allocation

Topic — getnode() Build123()

Topic — print function

Slide

THANK - YOU